

# Problem Set 1

## Applied Stats II

Due: February 11, 2026

### Instructions

- Please show your work! You may lose points by simply writing in the answer. If the problem requires you to execute commands in `R`, please include the code you used to get your answers. Please also include the `.R` file that contains your code. If you are not sure if work needs to be shown for a particular problem, please ask.
- Your homework should be submitted electronically on GitHub in `.pdf` form.
- This problem set is due before 23:59 on Wednesday February 11, 2026. No late assignments will be accepted.

### Question 1

The Kolmogorov-Smirnov test uses cumulative distribution statistics test the similarity of the empirical distribution of some observed data and a specified PDF, and serves as a goodness of fit test. The test statistic is created by:

$$D = \max_{i=1:n} \left\{ \frac{i}{n} - F_{(i)}, F_{(i)} - \frac{i-1}{n} \right\}$$

where  $F$  is the theoretical cumulative distribution of the distribution being tested and  $F_{(i)}$  is the  $i$ th ordered value. Intuitively, the statistic takes the largest absolute difference between the two distribution functions across all  $x$  values. Large values indicate dissimilarity and the rejection of the hypothesis that the empirical distribution matches the queried theoretical distribution. The p-value is calculated from the Kolmogorov- Smirnov CDF:

$$p(D \leq d) = \frac{\sqrt{2\pi}}{d} \sum_{k=1}^{\infty} e^{-(2k-1)^2\pi^2/(8d^2)}$$

which generally requires approximation methods (see Marsaglia, Tsang, and Wang 2003). This so-called non-parametric test (this label comes from the fact that the distribution of the test statistic does not depend on the distribution of the data being tested) performs

poorly in small samples, but works well in a simulation environment. Write an R function that implements this test where the reference distribution is normal. Using R generate 1,000 Cauchy random variables (`rcauchy(1000, location = 0, scale = 1)`) and perform the test (remember, use the same seed, something like `set.seed(123)`, whenever you're generating your own data).

As a hint, you can create the empirical distribution and theoretical CDF using this code:

```
1 # create empirical distribution of observed data
2 ECDF <- ecdf(data)
3 empiricalCDF <- ECDF(data)
4 # generate test statistic
5 D <- max(abs(empiricalCDF - pnorm(data)))
```

---

**Answer:** To evaluate whether the sample of 1,000 Cauchy random variables is consistent with a normal distribution, I implemented the Kolmogorov–Smirnov test manually in R. The test statistic is computed as the maximum absolute difference between the empirical CDF of the data and the theoretical CDF of a standard normal distribution.

The resulting KS statistic is:

$$D = 0.1347281$$

and the corresponding p-value, computed using the Kolmogorov distribution approximation, is:

$$p = 3.425363\text{e-}16$$

The extremely small p-value provides strong evidence against the null hypothesis that the data were drawn from a normal distribution. This result is expected, as the Cauchy distribution is heavy-tailed and differs substantially from the normal distribution.

#### R code used:

```
1 # KolmogorovSmirnov test function
2 myKS <- function(data) {
3
4   # sort data
5   x <- sort(data)
6   n <- length(x)
7
8   # empirical CDF
9   ECDF <- ecdf(x)
10  empiricalCDF <- ECDF(x)
11
12  # theoretical CDF under N(0,1)
13  theoreticalCDF <- pnorm(x)
```

```

14
15 # KS statistic
16 D <- max(abs(empiricalCDF - theoreticalCDF))
17
18 # Kolmogorov p-value approximation
19 k <- 1:100 # we approximate the infinite sum using the first 100 terms.
20 terms <- 2 * (-1)^(k-1) * exp(-2 * (k^2) * (D^2) * n)
21   # (-1)^(k-1) creates the alternating signs
22   # exp(-2 * (k^2) * (D^2) * n) are the exponential terms that shrink
   rapidly
23 p_value <- sum(terms)
24
25 return(list(
26   statistic = D,
27   p_value = p_value
28 ))
29 }
30
31 # generate data from Cauchy distribution
32 set.seed(123)
33 data <- rcauchy(1000, location = 0, scale = 1)
34
35 # run KS test
36 result <- myKS(data)
37 cat("KS Statistic:", result$statistic, "\np-value:", result$p_value)

```

## Question 2

Estimate an OLS regression in R that uses the Newton-Raphson algorithm (specifically BFGS, which is a quasi-Newton method), and show that you get the equivalent results to using `lm`. Use the code below to create your data.

---

### Answer:

To estimate the simple linear regression model using numerical optimisation, I implemented an OLS estimator via the BFGS quasi-Newton algorithm in R. The objective function minimises the sum of squared residuals, and the resulting coefficient estimates can be compared directly to those obtained from the closed-form OLS solution using `lm()`. Because OLS is the global minimiser of the squared-error loss, both approaches should yield equivalent parameter estimates.

The estimated coefficients from the BFGS optimisation and the standard `lm()` function are:

|                         | BFGS Estimate | lm() Estimate |
|-------------------------|---------------|---------------|
| Intercept ( $\beta_0$ ) | 0.1391778     | 0.1391874     |
| Slope ( $\beta_1$ )     | 2.7267000     | 2.7266985     |

The two methods produce numerically equivalent results, confirming that the BFGS algorithm successfully recovers the OLS estimates by minimising the sum of squared residuals.

### R code used:

```
1 set.seed(123)
2 data <- data.frame(x = runif(200, 1, 10))
3 data$y <- 0 + 2.75 * data$x + rnorm(200, 0, 1.5)
4
5 # Visualise generated data
6 plot(data$x, data$y, main="Scatter plot of y vs x", xlab="x", ylab="y")
7
8 # Define the sum of squared residuals (SSR) function
9 ssr <- function(par, x, y) {
10   beta0 <- par[1]
11   beta1 <- par[2]
12   y_hat <- beta0 + beta1 * x
13   sum((y - y_hat)^2)
14 }
15
16 # Use optim() with BFGS to minimise SSR
17 init_vals <- c(0, 0) # initial guesses for beta0 and beta1
18
19 fit_bfgs <- optim(
20   par = init_vals,
21   fn = ssr,
```

```

22 x = data$x,
23 y = data$y,
24 method = "BFGS"
25 )
26
27 fit_bfgs$par # estimated coefficients
28
29 # Compare with lm()
30 fit_lm <- lm(y ~ x, data = data)
31 coef(fit_lm)

```